

Christian Bucher - General Portfolio

Broad evidence-first portfolio overview.

Generated 2026-06-17

Core positioning

I want this package to be read as evidence of system building, not as a collection of screenshots. My strongest Fellowship signal is that I built infrastructure around models: remote command loops, a local control daemon, model routing, memory layers, voice control, evaluator loops, and claim boundaries.

This fits Secure AI because the hard work starts where AI systems leave chat and begin acting through tools, files, models, voice, and remote interfaces.

- Lead with the AI systems stack.
- Use ECSA as the future project direction, not as the main proof.
- Keep AMR, IP, and Quantum work bounded and honest.
- Show ambition through mechanism, not hype.

Best reading route

- 5 minutes: Start page, Remote WhatsApp-to-Agent, Miguel Core, Multi-Model Cascade.
- 20 minutes: AI Systems Dossier and Technical Architecture Appendix.
- Deep review: Future Project and the individual topic PDFs.

Remote WhatsApp-to-Agent Command Loop

Why I built it: A phone should be able to issue work to a home computer without turning the system into an unsafe direct shell. The design had to translate casual WhatsApp messages or audio into structured work orders, run them through a tool-enabled agent side, judge completion, and only then report back.

What I built / evidence: Implemented across OpenClaw watcher, MiguelAgent, EXOVIUM core, WA task manager, cognition routing, and architecture notes. Source notes document immediate ACK behavior, quality-loop thresholding, model routing, audio transcription, and two successful end-to-end tests.

Claim boundary: Prototype/evolved system. Some EXOVIUM pieces were later superseded by Miguel Core. Private channel identifiers and credentials are excluded.

Remote WhatsApp-to-Agent Command Loop - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- WhatsApp message or audio arrives through OpenClaw.
- A watcher normalizes inbound events, handles media, deduplicates repeated messages, and dispatches work.
- MiguelAgent interprets the request into goal, complexity, work order, success criteria, and model hint.

- The execution side uses EXOVIUM/Miguel tooling and model routing.
- A quality loop evaluates the result and can ask for an improved work order before replying.
- The final response is formatted and sent back through WhatsApp.

Remote WhatsApp-to-Agent Command Loop - what I can show

This is the clearest Secure AI signal: remote natural-language instructions became structured work orders, then passed through tool execution and evaluation before a user-facing reply.

- Independent design of a phone-to-computer agent workflow before relying on a managed product workflow.
- Separation of front-agent understanding/evaluation from back-agent execution.
- Ability to turn vague mobile input into structured work orders with success criteria.
- Awareness that autonomous work needs progress, interruption, deduplication, and post-work evaluation.
- Remote control is an authority problem, not just an interface problem.
- Acknowledge fast, execute slowly, and report only after evaluation.
- Audio, chat, browser events, model output, and tool execution all need different failure handling.
- Credentials and channel identifiers must be isolated from public documentation.

Why ECSA belongs here

ECSA is my future project answer: an AI-for-science direction where uncertainty, governance, validation, and model accountability matter. It should not replace the Secure AI infrastructure proof, but it gives the Fellowship a concrete research direction.

The architecture models antimicrobial resistance as a probabilistic graph/state-space problem with resistance, uncertainty, and stability rather than one static black-box prediction.

- Start with synthetic/public data before clinical data.
- Build Bayesian update behavior and graph learning under explicit uncertainty.
- Add governance, audit logs, model cards, and intended-use boundaries before real-world claims.
- Use lab-in-the-loop validation only after a reproducible proof of concept exists.